2021204042 황인성

2025 컴퓨터 그래픽 과제2

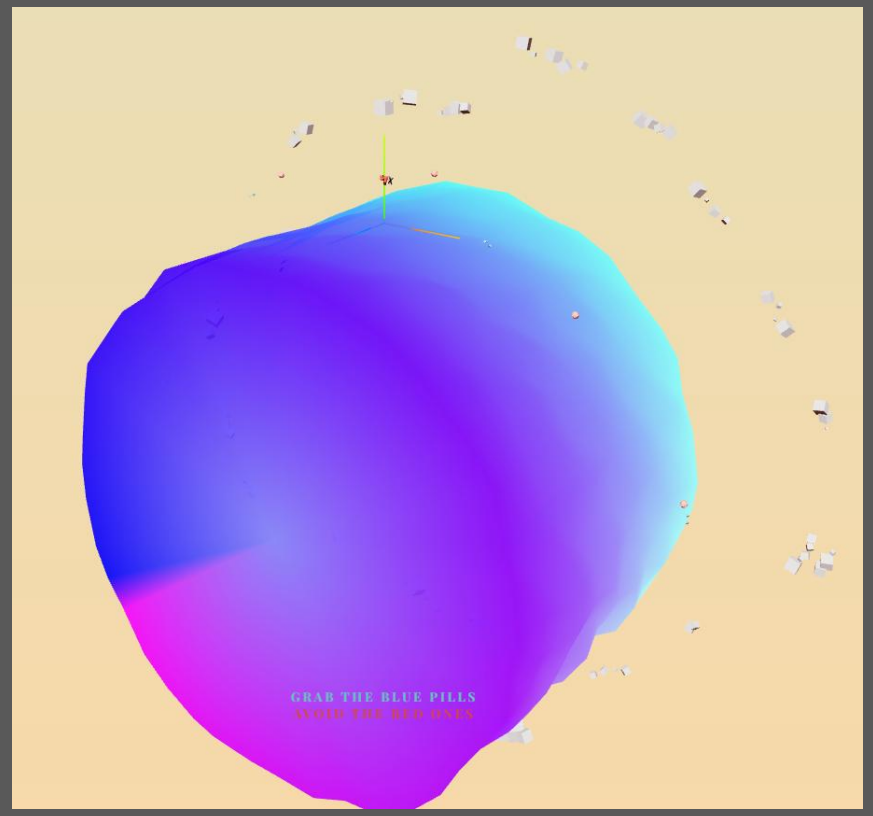
1. Vertex shader 에서 sea geometry 변형

구현 요약

0.Sea geometry 정점 재결합(1차 과제)

1.기존 코드 분석

2.셰이더에서 구현



1. Vertex shader 에서 sea geometry 변형

1차 과제에서 구현한 내용

같은 위치에 있는 vertex 들을 병합하여,

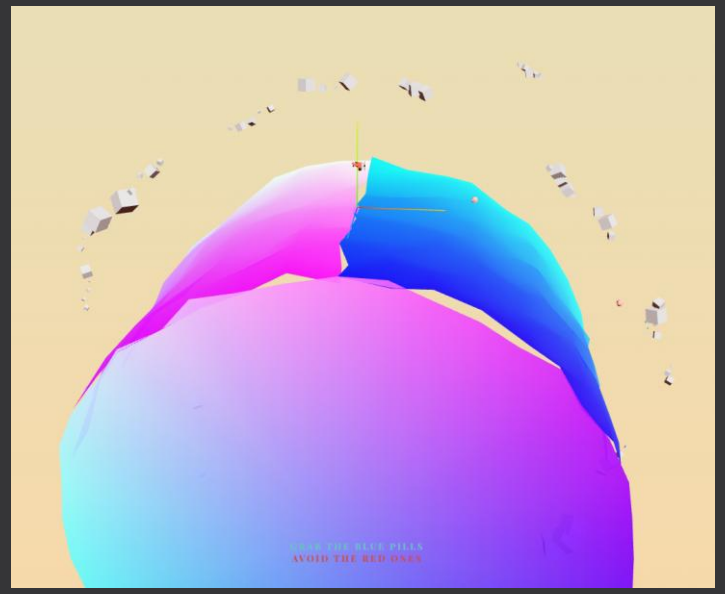
이후 정점들을 움직이거나 변형시키더라도

메시에 빈공간이 생기지 않게 함.

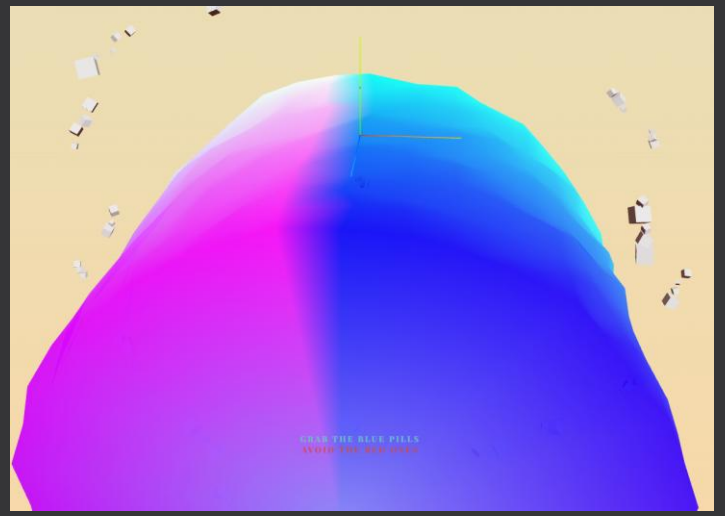
```
function Sea(){
  // 기본 geometry 생성
  let geom = new THREE.CylinderGeometry(game.seaR
  geom.rotateX(-Math.PI / 2);
  // mergeVertices 적용 (복사해서 새 geometry 생성)

  // 기존 코드
  // geom = mergeVertices(geom);

  // 오차 범위 명시
  geom = mergeVertices(geom,2.1);
```



적용 전



적용 후

1. Vertex shader 에서 sea geometry 변형

1.Sea 객체 생성 시, waves 배열에 각 정점 마다 정점의 위치/랜덤 각도/크기/속도 변수를 따로 저장합니다

```
기존 코드
this.waves = [];
for (let i = 0; i < vertexCount; i++) {
  const x = posAttr.getX(i);
  const y = posAttr.getY(i);
  const z = posAttr.getZ(i);
  this.waves.push({
    x,
    y,
    z,
    ang: Math.random() * Math.PI * 2,
    amp: game.wavesMinAmp + Math.random() * (game.wavesMaxAmp - game.wavesMinAmp),
    speed: game.wavesMinSpeed + Math.random() * (game.wavesMaxSpeed - game.wavesMinSpeed)
  });
}
```

```
Sea.prototype.moveWaves = function (){
  var verts = this.mesh.geometry.vertices;
  var l = verts.length;
  for (var i=0; i<l; i++){
    var v = verts[i];
    var vprops = this.waves[i];
    v.x =  vprops.x + Math.cos(vprops.ang)*vprops.amp;
    v.y = vprops.y + Math.sin(vprops.ang)*vprops.amp;
    vprops.ang += vprops.speed*deltaTime;
    this.mesh.geometry.verticesNeedUpdate=true;
  }
```

2.loop 내부에서 waves 배열 각 요소를 불러오고, 시간에 따라 angle 을 변화

cos sin 함수를 이용해 실제 Sea geometry 의 position 을 부드럽게 변형합니다

부드러운 변형 효과를 구현하기 위해선, 기존 코드의 waves 의 ang, amp 등, 각 정점마다 사용할 수 있는

시간에 따라 자연스럽게 변화시킬 수 있는 값이 필요합니다

waves 배열을 uniform 으로 전달하더라도 각 vertex shader는 for 문 처럼 따로 index가 존재하는 것이 아니기 때문에, 배열을 사용하지 못할 것 같습니다

따라서 vertex shader가 받는 정점의 정보를 활용하기로 결정했습니다.

1. Vertex shader 에서 sea geometry 변형

vertex shader가 받는 정보 중 position 은 각 정점의 위치를 나타냅니다.

따라서 이 값을 시드로 random 한다면,

시간에 상관없이 같은 정점은 같은 random 값을 가지게 됩니다.

이 값에 경과 시간을 더해서, 각 정점 마다 다르지만 시간에 따라 부드럽게 변화하는 값을 만들 수 있습니다.

```
let my_uniforms = {
  'my_color': { value: new THREE.Vector3(
  'wavesMin
  'wavesMax   (property) value: number
  'time': {value: 0.0}
  //'fogColor': { value: new THREE.Vector
```

```
Sea.prototype.moveWaves = function () {
  // CG:0513
  my_uniforms.time.value = performance.now()*0.001;
  //console.log(my_uniforms.time.value);
  return;
}
```

1. Vertex shader 에서 sea geometry 변형

하지만, Sea Mesh는 게임 스피드에 따라서 회전하므로,

같은 정점이라도 position이 이동하게 됩니다.

따라서 position 대신,

uv 좌표를 같은 방식으로 사용하였습니다.

```
let my_vtxShader = `
    //uniform mat4 modelViewMatrix;
    //uniform mat4 projectionMatrix;
    //attribute vec3 position;
    //attribute vec2 uv;
    uniform float time;
    uniform float wavesMinAmp;
    uniform float wavesMaxAmp;
    varying vec2 vUv;
    float random(vec2 st) {
        return fract(sin(dot(st.xy, vec2(12.9898,78.233))) * 43758.5453123);
    }
    void main()
    {
      vUv = uv;
      float ang = random(uv) * 3.14f * 2.f + time;  //시간에 따라 증가
      float amp = wavesMinAmp + random(uv) *  (wavesMaxAmp - wavesMinAmp);
      vec3 pos_new;
      pos_new.x = position.x + cos(ang) * amp;
      pos_new.y = position.y + sin(ang) * amp;
      pos_new.z = position.z;
      vec4 mvPosition = modelViewMatrix * vec4( pos_new, 1.0 );
      gl_Position = projectionMatrix * mvPosition;
    }
`;
```

2. 비행기, 카메라 조정

구현 요약

1.비행기, 카메라의 부모 객체 생성

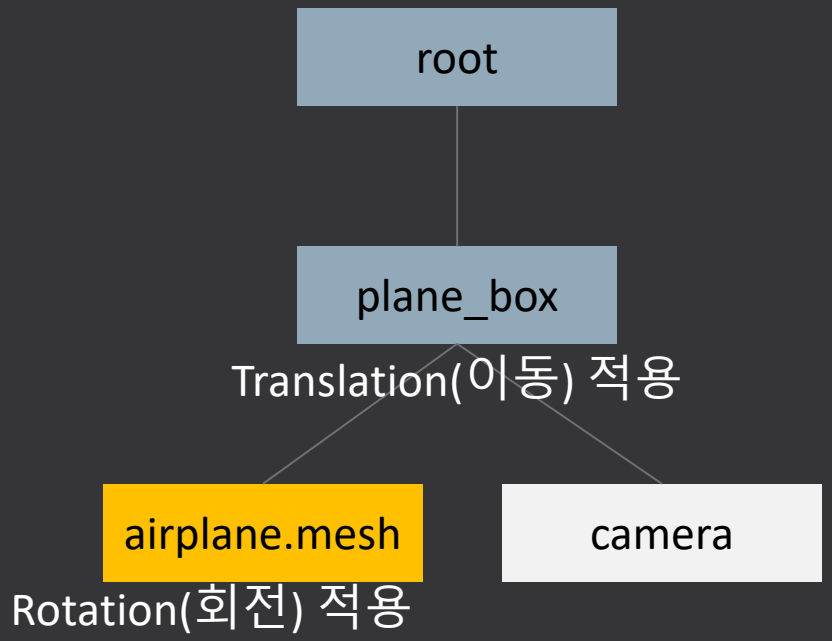
2.비행기 이동 구현

3.비행기 회전 구현

2. 비행기, 카메라 조정

기존 방식

새로운 방식




```
function createPlane(){
  const planeGeometry = new THREE.BoxGeometry(1, 1, 1);
  const blackMaterial = new THREE.MeshStandardMaterial({ color: 0x000000 });
  plane_box = new THREE.Mesh(planeGeometry, blackMaterial);
  plane_box.position.y = game.planeDefaultHeight;
  scene.add(plane_box);

  airplane = new AirPlane();
  airplane.mesh.scale.set(.25,.25,.25);
  //airplane.mesh.position.y = game.planeDefaultHeight;
  plane_box.add(airplane.mesh);
  //camera.lookAt(new THREE.Vector3(1000,0,0));
  plane_box.add(camera);
  //airplane.mesh.add(camera);
```

기존 camera가 airplane.mesh 의 자식으로 추가되는 방식 : airplane.mesh 의 이동과 회전 모두 camera에 적용됩니다

Airplane.mesh 와 camera 를 모두 자식으로 추가하는 plane_box 를 만들고,

이동은 plane_box에, 회전은 airplane.mesh에 적용하는 방식이 가능합니다

1.비행기, 카메라의 부모 객체 생성

2. 비행기, 카메라 조정

```
const gray_sphere_Geometry = new THREE.SphereGeometry(2, 32, 32);
const gray_Material = new THREE.MeshStandardMaterial({ color: 0x8
gray_sphere = new THREE.Mesh(gray_sphere_Geometry,gray_Material);
gray_sphere.position.y = game.planeDefaultHeight;
//gray_sphere.position.z = 100;
scene.add(gray_sphere);
}
```

```
plane_box.position.lerp(gray_sphere.position,0.05);
```

loop 에서, plain_box 는 gray_sphere 위치로 lerp

```
ocument.addEventListener("keydo
//console.log(e.code);
//KeyW KeyS KeyD KeyA
if (e.code === "KeyW") {
  gray_sphere.position.y+=300;
  rotate_time = rotate_speed;
  rotate_quat = new THREE.Quat
}
else if (e.code === "KeyS") {
  gray_sphere.position.y-=300;
  rotate_time = rotate_speed;
  rotate_quat = new THREE.Quat
}
else if (e.code === "KeyA") {
  gray_sphere.position.z-=300;
  rotate_time = rotate_speed;
  rotate_quat = new THREE.Quat
}
else if (e.code === "KeyD") {
  gray_sphere.position.z+=300;
```

키보드 입력에 따라, gray_sphere 위치 이동

2. 비행기, 카메라 조정

```
var rotate_speed = 100;
var rotate_time = 0;
var rotate_quat;
```

```
document.addEventListener("keydown", (e) => {
  //console.log(e.code);
  //KeyW KeyS KeyD KeyA
  if (e.code === "KeyW") {
    gray_sphere.position.y+=300;
    rotate_time = rotate_speed;
    rotate_quat = new THREE.Quaternion().setFromEuler(new THREE.Euler(0, 0, Math.PI/1.5));
  }
  else if (e.code === "KeyS") {
    gray_sphere.position.y-=300;
    rotate_time = rotate_speed;
    rotate_quat = new THREE.Quaternion().setFromEuler(new THREE.Euler(0, 0, -Math.PI/1.5));
  }
  else if (e.code === "KeyA") {
    gray_sphere.position.z-=300;
    rotate_time = rotate_speed;
    rotate_quat = new THREE.Quaternion().setFromEuler(new THREE.Euler(-Math.PI/1.5, 0, 0));
  }
  else if (e.code === "KeyD") {
    gray_sphere.position.z+=300;
    rotate_time = rotate_speed;
    rotate_quat = new THREE.Quaternion().setFromEuler(new THREE.Euler(Math.PI/1.5, 0, 0));
  }
});
```

키보드 입력을 받으면 count down 초기화,

목표 쿼터니언 설정

```
if(rotate_time>0){
  rotate_time--;
  if(rotate_time>=rotate_speed/2){
    airplane.mesh.quaternion.slerp(rotate_quat,0.1);
  }
  else {
    airplane.mesh.quaternion.slerp(new THREE.Quaternion().setFromEuler(new THREE.Euler(0, 0, 0)),0.1);
  }
}
```

Loop 에서 반복

남은 시간이 절반 이상 :

airplain.mesh 를 목표 쿼터니언으로 slerp

남은 시간이 절반 이하 :

airplain.mesh 를 기본 쿼터니언으로 slerp